

好，我们正式进入 **M2：函数、参数、传值、引用、const、重载、signature**。

这一层比第一层抽象一点，但它其实只围绕一个核心问题：

**函数被调用时，它里面看到的那个名字，到底是“外面数据的一份独立值”，还是“外面那个对象本身的另一个名字”？**

一旦这句话搞懂，`int x`、`int& x`、`const int& x` 基本就通了。承接文件要求这一层先讲函数、parameter/argument、声明/定义/返回值，再讲传值和引用，不能直接把 & 丢给你背。

CSC3002\_Exam\_First\_Teaching\_Han...

## 1. 函数 function 本身是什么？

### 第一件事：函数是什么？

函数 (function) 是一段被命名的代码，它可以接收一些输入，执行一系列语句，并且可以把一个结果返回给调用它的地方。

例如：

```
int add(int a, int b) {  
    int result = a + b;  
    return result;  
}
```

这个东西就是一个函数。

C++ 的正式描述也是：函数把一组语句 (function body) 与函数名以及零个或多个参数关联起来；调用时参数由实参初始化，然后执行函数体。Cppreference

## 2. 为什么要有函数？

假设没有函数，你每次想算：

两个数相加

都重新写一遍代码。

有函数以后，可以先定义：

```
int add(int a, int b) {  
    return a + b;  
}
```

以后直接：

`add(3, 5)`

或者：

`add(10, 20)`

所以函数主要解决：

**把一段有明确功能的操作包装起来，可以重复调用，而且调用者只需要知道怎么提供输入、会得到什么输出。**

以后 Assignment 1 里的：

```
str_len(...)
str_clear(...)
upper(...)
operator+(...)
```

从根本上说也都是“提供某个操作的函数”。

### 3. 一个函数到底由什么组成？

看：

```
int add(int a, int b) {
    return a + b;
}
```

拆开：

|      |     |                |         |
|------|-----|----------------|---------|
| int  | add | (int a, int b) | { ... } |
| ↑    | ↑   | ↑              | ↑       |
| 返回类型 | 函数名 | 参数列表           | 函数体     |

一个一个定义。

### 4. 返回类型 return type 是什么？

#### 第一件事

**返回类型就是函数调用完成以后，它向调用者提供的结果是什么类型。**

例如：

```
int add(int a, int b)
```

最前面的：

```
int
```

表示：

`add(...)` 最终提供一个 `int` 类型的结果。

所以：

```
add(3, 5)
```

本身也是一个**表达式**。

这和第一层立刻连起来了：

表达式：  
`add(3, 5)`

类型：  
`int`

值：  
`8`

于是可以：

```
int x = add(3, 5);
```

过程就是：

```
add(3,5)
↓
int 8
↓
初始化 x
↓
x = 8
```

## 5. return 本身是什么？

### 第一件事

**return** 是结束当前函数调用，并把结果交还给调用者的语句。

例如：

```
int square(int x) {
    return x * x;
}
```

调用：

```
square(4)
```

函数内部：

```
x = 4
↓
x * x = 16
↓
return 16
```

所以整个：

```
square(4)
```

表达式的结果：

```
类型: int  
值: 16
```

## 6. void 是什么?

你马上会看到:

```
void change(int x)
```

所以必须先定义。

### void 本身是什么?

在函数返回类型的位置:

**void** 表示这个函数不向调用者返回一个普通的结果值。

例如:

```
void printHello() {  
    cout << "Hello";  
}
```

它可以做事情:

```
输出 Hello
```

但是你不能像:

```
int x = printHello();
```

这样要求它提供一个 `int`。

## 7. 参数 parameter 是什么?

### 第一件事

**参数 (parameter)** 是写在函数定义里的、用来接收每次调用所提供数据的名字。

例如:

```
int add(int a, int b)
```

这里:

```
a  
b
```

就是 parameters。

它们的类型分别是:

```
int  
int
```

## 8. 实参 argument 是什么？

这个词和 parameter 很容易混。

### argument 本身是什么？

**实参 (argument) 是在真正调用函数时，你实际提供进去的值或表达式。**

比如：

```
int x = 3;  
int y = 7;  
  
add(x, y);
```

函数定义：

```
int add(int a, int b)
```

那么：

```
parameter (形参) :  
a, b  
  
argument (实参) :  
x, y
```

特别重要：

```
a 和 b 是函数写出来时就存在于定义中的角色。  
  
x 和 y 是这一次调用真正传进去的东西。
```

C++ 的函数调用规则正是：调用发生时，parameters 根据 arguments 被初始化。 [Cppreference](#)

## 9. 用一次完整调用看看

```
int add(int a, int b) {  
    int result = a + b;  
    return result;  
}  
  
int main() {  
    int x = 3;  
    int y = 7;  
  
    int z = add(x, y);  
}
```

调用:

```
add(x, y)
```

此时:

```
argument x 当前值 = 3  
argument y 当前值 = 7
```

函数参数:

```
a 得到 3  
b 得到 7
```

然后:

```
int result = a + b;
```

得到:

```
result = 10
```

然后:

```
return result;
```

所以:

```
add(x, y)
```

产生:

```
int 10
```

最后:

```
int z = 10;
```

所以:

```
z = 10
```

---

## 10. 什么是函数调用 function call?

### 第一件事

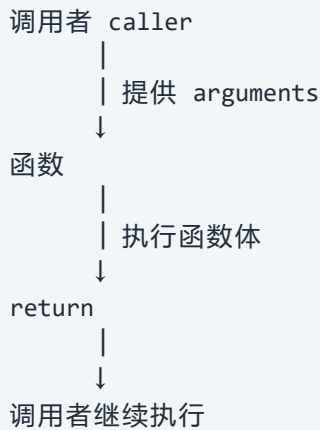
**函数调用**, 就是要求程序执行某个函数, 并为它提供这一次使用的 **arguments**。

例如:

```
add(3, 5)
```

就是一次 function call。

你可以粗略想象：



以后我们经常说：

- caller: 调用者
- callee: 被调用的函数

现在只需要认识即可。

## 11. 函数内部的变量和外面的变量是不是一个东西？

现在真正进入第二层的核心。

看：

```
void change(int x) {  
    x = 100;  
}  
  
int main() {  
    int a = 5;  
  
    change(a);  
  
    cout << a;  
}
```

你第一眼可能会想：

我把 a 给了 change，里面 x=100，那 a 是不是也变 100？

答案：

不是。

为什么？

必须先解释 **传值**。

## 12. 什么是传值 pass by value?

### 第一件事

在我们当前最简单的 `int` 情况下：

**传值**，就是函数参数拥有一个独立的对象；调用时，它根据 `argument` 的值进行初始化。

例如：

```
void change(int x)
```

这里的：

```
int x
```

是函数自己的参数对象。

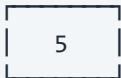
如果调用：

```
int a = 5;  
change(a);
```

概念上可以画：

main 中：

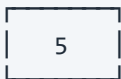
a



调用 `change(a)`：

`change` 内部出现另一个 `int`：

x



注意：

a 和 x 当前值一样

但：

a 和 x 不是同一个对象

这就是核心。

## 13. 所以 `x = 100` 改的是谁？

函数：

```
void change(int x) {  
    x = 100;  
}
```

刚进入时：

```
a = 5  
x = 5
```

然后：

```
x = 100;
```

得到：

```
a = 5  
x = 100
```

函数结束以后，那个参数 `x` 不再需要。

外面的：

```
a
```

从头到尾没有被修改。

所以输出：

```
5
```

## 14. 不要把函数理解成“代码文本直接粘过去”

这是一个很重要的模型。

你不能把：

```
change(a);
```

简单理解为：

```
把函数体里的 x 全部换成 a。
```

如果真这么理解，你就会错误地认为：

```
x = 100;
```

相当于：

```
a = 100;
```

但传值参数不是这样。

课程材料也是通过函数调用状态和 `swap` 示例解释为什么 `pass-by-value` 不改变调用者变量。

CSC3002\_Exam\_First\_Teaching\_Han...

## 15. 什么是一次函数调用的“局部状态”？

你以后课件里会看到 stack frame。

### stack frame 在这里是什么？

当前课程模型里，你可以先理解成：

**每调用一次函数，程序会为这次调用维护一套自己的参数、局部变量以及返回位置等调用状态。**

例如：

```
int add(int a, int b) {  
    int result = a + b;  
    return result;  
}
```

调用时概念上：

```
main 的状态  
-----  
x  
y  
z  
  
      ↓ 调用 add  
  
add 这一次调用的状态  
-----  
a  
b  
result
```

函数结束后，这次调用状态结束。

这是非常有用的理解模型。

但先别把它误解成：

C++ 标准保证物理内存一定按我们画的矩形存放。

实际编译器可以优化。

我们目前用它理解**语言语义和调用过程**。

## 16. 为什么传值会让 swap 失败？

经典例子：

```
void swapBad(int a, int b) {  
    int temp = a;  
    a = b;  
    b = temp;  
}
```

然后：

```
int x = 3;
int y = 8;

swapBad(x, y);
```

开始:

```
main:
x = 3
y = 8
```

调用后参数:

```
swapBad:
a = 3
b = 8
```

执行:

```
int temp = a;
```

得到:

```
temp = 3
```

然后:

```
a = b;
```

得到:

```
a = 8
```

然后:

```
b = temp;
```

得到:

```
b = 3
```

所以函数内部成功变成:

```
a = 8
b = 3
```

但 main 里:

```
x = 3
y = 8
```

完全没变。

因为换的是:

两个参数对象的值

而不是：

调用者的两个原对象。

## 17. 那如果函数真的需要修改外面的变量怎么办？

这时候才进入：

### reference 引用

#### 第一件事：引用本身是什么？

引用（reference）是一个已经存在对象的别名（alias）。

“别名”不要当成新术语继续循环。

它的意思就是：

同一个对象，可以用另一个名字去访问。

例如：

```
int x = 5;  
int& r = x;
```

这里：

x

是原来的名字。

r

是对同一个 `int` 对象的引用。

C++ 的引用声明就是把名字声明为一个已经存在对象或函数的 `alias`。Cppreference

## 18. `int& r = x` 到底逐个符号是什么意思？

我们严格拆。

```
int& r = x;
```

`int`

说明：

r 引用的是一个 `int`。

`&`

这里出现在声明里，表示：`r` 是引用。

所以：

```
int& r
```

读作：

`r` 是一个 `int` 的引用。

现在不要把这里的 `&` 和 “取地址” 混起来。

那是 M3 的另一个语境。

当前：

```
int& r
```

里的 `&`：

**reference declaration.**

```
= x
```

表示：

让这个引用 `r` 绑定 (bind) 到现有对象 `x`。

所以：

```
int x = 5;  
int& r = x;
```

不是：

`x` 有一个 5  
`r` 又复制一个 5

而是：



这张图非常重要。

## 19. 修改 `r` 会发生什么？

```
int x = 5;  
int& r = x;
```

```
r = 10;
```

因为：

`r` 是 `x` 那个对象的另一个名字

所以：

```
r = 10;
```

修改的其实就是同一个对象。

结果：

```
x = 10
r = 10
```

不是“r 改了以后同步给 x”。

这个说法容易误导。

更准确是：

**本来就只有一个被引用的 `int` 对象，只是可以通过 `x` 或 `r` 两个名字访问它。**

## 20. 引用能不能以后改去引用另一个对象？

看：

```
int x = 5;
int y = 8;

int& r = x;
```

此时：

`r` 引用 `x`

然后：

```
r = y;
```

很多初学者会猜：

`r` 现在改去引用 `y` 了。

不是。

引用一旦初始化绑定，不能通过普通赋值“重新绑定”到另一个对象。[Cppreference](#)

这里：

```
r = y;
```

实际上等于：

把 `y` 的值赋给 `r` 所引用的对象。

因为 `r` 引用 `x`：

```
x = 8
y = 8

r 仍然引用 x
```

这一点以后和指针差别很大。

## 21. 引用参数是什么？

现在终于回到：

```
void change(int& x)
```

### 第一件事

引用参数，就是函数的 `parameter` 本身是一个引用；调用时，它绑定到调用者提供的那个对象。

例如：

```
void change(int& x) {  
    x = 100;  
}
```

调用：

```
int a = 5;  
change(a);
```

现在和传值完全不同。

### 传值版本

```
void change(int x)
```

模型：

外面：

a



函数里面：

x

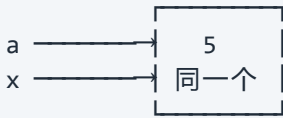


两个独立 `int` 对象

### 引用版本

```
void change(int& x)
```

模型：



这里函数参数名字：

x

直接引用调用者的：

a

所以：

```
x = 100;
```

直接把那个共同对象改成：

100

于是函数结束以后：

```
a = 100
```

## 22. 两个 change 对照

这是这一层最重要的对照。

### 版本 A：传值

```
void change(int x) {
    x = 100;
}

int main() {
    int a = 5;
    change(a);
    cout << a;
}
```

输出：

5

因为：

x 是独立参数对象

## 版本 B: 传引用

```
void change(int& x) {  
    x = 100;  
}  
  
int main() {  
    int a = 5;  
    change(a);  
    cout << a;  
}
```

输出:

```
100
```

因为:

x 是 a 的引用

这就是你目前最应该真正吃透的一对代码。

## 23. 用引用重新实现 swap

现在:

```
void swapGood(int& a, int& b) {  
    int temp = a;  
    a = b;  
    b = temp;  
}
```

调用:

```
int x = 3;  
int y = 8;  
  
swapGood(x, y);
```

绑定关系:

参数 a → 外面的 x  
参数 b → 外面的 y

所以:

```
int temp = a;
```

得到:

```
temp = 3
```

然后:

```
a = b;
```

因为：

```
a → x  
b → y
```

所以相当于：

```
x = y 的值
```

得到：

```
x = 8
```

再：

```
b = temp;
```

相当于：

```
y = 3
```

最后：

```
x = 8  
y = 3
```

这就是 reference parameter 最直接的用途之一。

## 24. 那为什么函数不全部都用引用？

因为函数有时候根本就不应该修改调用者的数据。

例如：

```
int square(int x) {  
    return x * x;  
}
```

你只是想：

```
给我一个数，我算平方。
```

根本没必要修改外面的对象。

所以：

```
int x
```

非常合理。

另一方面，有些函数的目的就是修改调用者：

```
void increment(int& x) {  
    ++x;  
}
```

那 reference 很合理。

还有第三种情况：

我不想复制一个大对象，但是也不想允许函数修改它。

这就是：

## const T&

## 25. const 本身是什么？

一定先单独定义。

### const 是什么？

**const** 是类型限定符 (type qualifier)，用来表达：通过这种受 **const** 限定的访问方式，不允许修改对象。

最简单：

```
const int x = 5;
```

这里：

x 的类型：  
const int

你不能正常写：

```
x = 10;
```

因为你把 x 声明成了不能被修改的 const int。

C++ 把 const 作为类型限定符使用。Cppreference

## 26. const int& x 要怎样逐层读？

现在组合三个已经定义的东西：

```
const int& x
```

千万不要整坨背。

先：

```
int
```

→ 整数类型。

再：

```
const int
```

→ 不允许通过这个 `const int` 视角进行修改的整数。

再：

```
const int&
```

→ 对 `const int` 的引用。

所以函数：

```
void show(const int& x)
```

意思可以读作：

`x` 是一个引用，它可以让我访问调用者的整数，但这个函数不能通过 `x` 去修改那个整数。

## 27. `const int&` 会让原对象永远变成 `const` 吗？

不会。

例如：

```
void show(const int& x) {  
    cout << x;  
}
```

外面：

```
int a = 5;  
  
show(a);  
  
a = 10;
```

完全可以。

因为原对象：

```
a
```

本身是：

```
int
```

不是：

```
const int
```

只是函数内部这个访问路径：

```
x
```

是：

```
const int&
```

所以函数内部不能：

```
x = 100;    // 不允许
```

但函数结束以后，调用者当然仍然可以：

```
a = 100;
```

这点以后非常重要：

`const T&` 通常意味着“我借来看看，但不通过这个引用修改你”。

## 28. int、int&、const int& 三兄弟

现在可以把三种参数真正对齐：

| 参数                            | 函数拿到什么模型                | 函数能否改调用者原对象 |
|-------------------------------|-------------------------|-------------|
| <code>int x</code>            | 独立的 <code>int</code> 参数 | ✗           |
| <code>int&amp; x</code>       | 调用者对象的引用                | ✓           |
| <code>const int&amp; x</code> | 调用者对象的只读引用              | ✗ 通过这个引用不行  |

这是第二层最核心的表。

## 29. 为什么 Assignment 1 会大量出现 `const ...&`?

这和作业非常直接。

承接文件记录了 A1 中一些行为：

- `upper/lower` 返回新的 `str`，**不修改输入**
- `+` 返回新字符串，**不修改操作数**
- `+=` 修改左操作数，并且返回其引用。CSC3002\_Exam\_First\_Teaching\_Han...

你现在已经可以理解背后的思想了。

例如概念上：

```
something(const str& input)
```

表达：

我要访问 `input`，但这个函数不应该通过它修改输入对象。

而：

```
something(str& input)
```

则可能表达：

```
这个函数有权修改这个 str。
```

至于 A1 **真实函数签名具体长什么样**，承接文件明确说源文件 `string.h` 还没有收到，所以现在不能替老师编造完整接口。CSC3002\_Exam\_First\_Teaching\_Han...

这一点我们会守住。

## 30. 什么是函数声明 declaration?

现在再补函数组织。

### 第一件事

**函数声明 (function declaration)** 告诉编译器：有这么一个函数，它叫什么、返回什么类型、接收什么参数类型。

例如：

```
int add(int a, int b);
```

注意最后：

```
;
```

这里只有接口信息。

没有：

```
{
    ...
}
```

也没有真正函数体。

## 31. 什么是函数定义 definition?

### 第一件事

**函数定义 (function definition)** 真正给出这个函数做什么，也就是提供函数体。

例如：

```
int add(int a, int b) {
    return a + b;
}
```

所以：

```
declaration
= 告诉别人 存在这样一个函数”

definition
= 真正告诉程序 这个函数具体怎么做”
```

这以后就是 `.h / .cpp` 的基础。

例如概念上：

```
// math.h
int add(int a, int b);
```

然后：

```
// math.cpp
int add(int a, int b) {
    return a + b;
}
```

这个 “interface / implementation” 分工在你们课程和 A1 都非常重要。

CSC3002\_Exam\_First\_Teaching\_Han...

## 32. 什么是 function signature?

### 第一件事

**函数签名 (function signature) 是一组用来标识和区分函数的关键信息。**

对我们当前课程里的普通非成员函数，你最重要的是：

**函数名 + parameter 的类型列表。**

例如：

```
int add(int a, int b)
```

基础 signature 可以关注：

```
add(int, int)
```

而不是：

```
a
b
```

参数名字不是区分函数 signature 的关键部分。

C++ 对普通函数 signature 的规范定义包括函数名和 parameter-type-list，并涉及其所属 namespace；成员函数还有额外限定信息。[Cppreference](#)

课程当前阶段先用：

**name + parameter types**

就足够了。

## 33. 返回类型算不算普通函数 signature?

当前课程最重要的结论：

**不能只靠返回类型不同来重载普通函数。**

例如：

```
int f(int x);  
double f(int x);
```

这是不允许通过重载来区分的。

因为调用：

```
f(5)
```

编译器不能仅仅因为你将来想拿 `int` 或 `double`，就把它们视作合法的两个 overload。

cppreference 也明确指出：仅返回类型不同的函数不能构成这样的重载。[Cppreference](#)

## 34. 什么是 function overloading?

### 第一件事

**函数重载 (function overloading) 就是：**在同一个作用域中，可以有多个同名函数，只要它们的参数形式足够不同，使编译器能够根据调用找到合适版本。

例如：

```
int twice(int x) {  
    return x * 2;  
}  
  
double twice(double x) {  
    return x * 2;  
}
```

现在有两个函数都叫：

```
twice
```

但是：

```
twice(int)  
twice(double)
```

signature 不同。

## 35. 调用时编译器怎么选?

例如：

```
twice(5)
```

5 是：

```
int
```

所以最匹配：

```
twice(int)
```

而：

```
twice(5.5)
```

5.5 是：

```
double
```

所以最匹配：

```
twice(double)
```

当一个函数名对应多个 overload 时，编译器会进行 overload resolution，在候选函数中选择最匹配 argument 的可行版本。[Cplusplusreference](#)

所以可以画：

```
twice(5)
  |
  | argument = int
  ↓
twice(int)
```

而：

```
twice(5.5)
  |
  | argument = double
  ↓
twice(double)
```

## 36. 参数名字不同算不算重载？

不算。

例如：

```
void f(int x);
void f(int y);
```

不是两个 overload。

它们本质上都是：

```
f(int)
```

参数叫什么：

```
x  
y  
abc  
hello
```

不会改变这个基础 signature。

## 37. 参数数量不同呢？

可以。

例如：

```
int add(int a, int b);  
  
int add(int a, int b, int c);
```

可以同时存在。

因为：

```
add(int, int)  
  
add(int, int, int)
```

不同。

调用：

```
add(2, 3);
```

选第一个。

调用：

```
add(2, 3, 4);
```

选第二个。

## 38. 参数类型不同呢？

也可以。

```
void print(int x);  
void print(double x);
```

即：

```
print(int)  
print(double)
```

这是两个 overload。

## 39. 为什么 C++ 要支持 overload?

因为很多概念操作是一样的，但数据类型不同。

例如：

```
abs(...)
```

你可能希望：

```
abs(-3)
```

也能用。

```
abs(-3.5)
```

也能用。

调用者只需要记：

```
abs
```

编译器根据参数类型找到合适版本。

而 Assignment 1 的 operator overloading，本质上也是这套思想再往前走一步：

**让 +、+=、<< 这些已有运算符，对你自己定义的类型拥有适当行为。**

所以 M2 的 ordinary function overload 是 M8 operator overload 的直接前置。

## 40. 还有一个课件里的 default argument

你的 M2 范围其实还包括这个，我们不能因为你刚才没列出来就偷偷跳过。承接文件明确把 L2 的默认参数纳入函数部分。CSC3002\_Exam\_First\_Teaching\_Han...

### default argument 是什么?

**默认实参 (default argument) 就是：如果调用者没有显式提供某个 argument，函数声明预先规定一个默认值。**

例如：

```
void greet(int times = 1) {
    // ...
}
```

那么：

```
greet();
```

可以使用：

```
times = 1
```

而：

```
greet(3);
```

则：

```
times = 3
```

当前只需要知道这是什么。

不需要现在展开所有默认参数规则。

## 41. 现在把整个函数调用模型拼起来

看这一段：

```
int multiply(int a, int b) {  
    return a * b;  
}  
  
int main() {  
    int x = 3;  
    int y = 4;  
  
    int z = multiply(x, y);  
}
```

你的脑子应该这样运行：

### ① 找函数

```
multiply
```

### ② 看 arguments

```
x  
y
```

当前：

```
x = 3  
y = 4
```

### ③ 看 parameters

```
int a  
int b
```

这是 pass-by-value。

所以：

```
a 是独立 int 参数，初始值 3  
b 是独立 int 参数，初始值 4
```

#### ④ 执行函数体

```
a * b
```

得到:

```
int 12
```

#### ⑤ return

整个:

```
multiply(x, y)
```

成为:

```
类型 int  
值 12
```

#### ⑥ 初始化 z

```
z = 12
```

这就把 M1:

```
表达式有类型和值
```

和 M2:

```
函数调用产生结果
```

彻底连接起来了。

## 42. 再把引用调用模型拼起来

```
void addOne(int& x) {  
    ++x;  
}  
  
int main() {  
    int n = 5;  
    addOne(n);  
}
```

你的脑子应该:

#### ① argument

```
n
```

是一个现有 `int` 对象。

## ② parameter

```
int& x
```

不是创建一个独立 `int` 副本。

而是：

```
x 引用 n
```

## ③ 执行

```
++x;
```

因为 `x` 和 `n` 表示同一个被引用对象：

```
n: 5 → 6
```

所以函数结束：

```
n = 6
```

## 43. 现在你真正要掌握的是这四句话

如果这四句能自己讲出来，第二层主模型就基本成立了：

### ①

```
void f(int x)
```

`x` 是一个传值参数；在简单 `int` 情况下，它是独立参数对象，根据 `argument` 的值初始化。

### ②

```
void f(int& x)
```

`x` 是引用参数，它引用调用者提供的对象，所以通过 `x` 修改会影响原对象。

### ③

```
void f(const int& x)
```

`x` 是一个只读引用参数；它引用对象，但函数不能通过这个引用修改对象。

### ④

```
int f(int x)
```

最前面的 `int` 是返回类型，表示函数调用产生一个 `int` 结果。

## 44. 和 Assignment 1 的关系现在已经开始非常直接

你们 A1 的任务目标明确包括：

- references
- function overloading
- operator overloading
- assignment
- constructors
- streams
- namespaces。CSC3002\_Exam\_First\_Teaching\_Han...

而行为里又有：

```
upper/lower
→返回新对象，不修改输入

+
→返回新对象，不修改 operands

+=
→修改左 operand，并返回其引用
```

CSC3002\_Exam\_First\_Teaching\_Han...

所以你今天学的：

```
T
T&
const T&
return T
```

不是孤立语法。

后面会直接变成：

```
这个函数应该不应该修改输入？

它应该返回一个新对象还是原对象？

为什么 += 返回引用？
```

等实际接口设计问题。

## 45. A1 到底还差哪些层？

现在给你一个非常具体的进度地图。

假设今天第二层学完：

```
✓ M1 类型、值、表达式
← 正在学 M2 函数、引用
```

为了能够理解并自己完成 A1，接下来主要还要：

| 模块                                    | A1 为什么需要         |
|---------------------------------------|------------------|
| M3 基本对象/地址/指针                         | 看懂底层对象关系和字符处理    |
| M4 数组/容量/越界                           | data 是固定容量字符存储   |
| M6 char / C-style string / \0         | 作业核心数据表示         |
| M7 struct、constructor、对象              | mystring::str 本身 |
| M8 overload、operator、stream、namespace | 作业接口本体           |

所以还有东西，但不是：

“我还得学半本 C++ 才能开始作业。”

实际上承接文件判断这个 Assignment 的难点主要是：

**有限长度字符序列 + 类型和接口 + 引用/const/构造/运算符 + 边界。**

而不是高级算法。CSC3002\_Exam\_First\_Teaching\_Han...

考虑到承接文件记录的截止时间是 **2026 年 10 月 8 日 23:59**，我们学完 M2 后应该直接走作业急用路线，而不是先去把 Q31 那套复杂指针自增练完。CSC3002\_Exam\_First\_Teaching\_Han...

## 46. 现在你做两道，不要运行

先检验最核心的 **传值 vs 引用**。

### 题 1

```
void change(int x) {
    x = x + 10;
}

int main() {
    int a = 5;
    change(a);
    cout << a;
}
```

回答：

调用 change(a) 时：  
argument 是谁？  
parameter 是谁？  
x 和 a 是不是同一个对象？  
  
函数内部 x 最后等于？  
函数结束后 a 等于？  
最终输出？

## 题 2

只改一个字符：

```

void change(int& x) {
    x = x + 10;
}

int main() {
    int a = 5;
    change(a);
    cout << a;
}

```

回答：

这里 `int& x` 本身是什么意思？

调用 `change(a)` 后：

`x` 引用谁？

执行 `x = x + 10` 后，谁被修改？

`a` 最后等于？

输出？

最后再判断下面三句话：

```

void f(int x);
void f(double x);
void f(int& x);

```

A. 前两个能否构成 overload？为什么？

B.

```

int f(int x);
double f(int x);

```

能不能仅靠返回类型不同形成 overload？

C.

```

void show(const int& x)

```

函数内部能不能写：

```

x = 20;

```

为什么？

你把这几题答完，我们就能判断第二层主模型是不是已经建立。如果稳定，下一步我建议直接进入 **M3 的最基础部分：对象 → 存储位置 → 地址 → 指针到底是什么**，然后迅速接 M4/M6，为 A1 铺路。

## 附录：第 46 节题目解析（新增）

本附录是新增内容。前面的讲解、例子、题目、链接及原文中的资料简写均按原文保留；这里只补上文末两道代码题和 A/B/C 三个判断的解析。下文沿用原文的代码片段语境，假定输出语句所需的头文件及命名空间已准备好。两道 `change` 题是分别分析的程序。

### 题 1：传值参数，最终输出 5

#### 1. 先看题目到底在考什么

关键不是“有没有把 `a` 交给函数”，而是：**函数中的 `x`，是独立的参数对象，还是 `a` 所表示对象的另一个名字？**

题目写的是：

```
void change(int x)
```

这里 `int x` 表示传值参数。在本题的简单 `int` 情况下，调用时会用实参的值初始化一个独立的参数对象 `x`。

#### 2. 逐个确定角色

| 问题                                        | 答案                     | 为什么                                                   |
|-------------------------------------------|------------------------|-------------------------------------------------------|
| argument 是谁？                              | 调用表达式中的 <code>a</code> | <code>change(a)</code> 的括号里实际提供的是 <code>a</code>      |
| parameter 是谁？                             | 函数中的 <code>x</code>    | 函数参数声明写的是 <code>int x</code>                          |
| <code>x</code> 和 <code>a</code> 是不是同一个对象？ | 不是                     | <code>x</code> 是独立的传值参数对象                             |
| 函数内部 <code>x</code> 最后等于？                 | 15                     | <code>x</code> 先由 5 初始化，再执行加 10                       |
| 函数结束后 <code>a</code> 等于？                  | 5                      | 函数只修改了独立的 <code>x</code>                              |
| 最终输出？                                     | 5                      | <code>cout &lt;&lt; a</code> 读取的是调用者中的 <code>a</code> |

#### 3. 按执行顺序推演

第一步：

```
int a = 5;
```

调用者中有一个 `int` 对象，名字是 `a`，当前值是 5。

第二步：

```
change(a);
```

实参表达式 `a` 的值是 5。用这个值初始化函数的独立参数对象 `x`。

调用者：`a = 5`

函数内：`x = 5`

两个对象，当前值相同。

第三步：

```
x = x + 10;
```

右边  $x + 10$  先读取参数  $x$  的值 5，算出 15；左边的赋值再把 15 存入参数对象  $x$ 。

```
调用者: a = 5
函数内: x = 15
```

第四步：函数结束，参数  $x$  的这次使用结束。`void` 表示函数不提供普通返回值；本题也没有把任何结果赋回  $a$  的语句。

第五步：

```
cout << a;
```

读取  $a$ ，所以输出：

```
5
```

#### 4. 为什么容易误判成 15?

如果把 `change(a)` 当成“把函数体里的  $x$  全部替换成  $a$ ”，就会误以为执行的是  $a = a + 10$ 。本题实际创建并修改的是独立的参数对象  $x$ 。

需要记住的判断顺序是：**先看参数声明，再判断对象关系，最后计算值。**

### 题 2：引用参数，最终输出 15

#### 1. 只多了一个 &，对象关系就变了

题目写的是：

```
void change(int& x)
```

逐项读：

- `void`：这次函数调用不提供普通返回值。
- `int`：被引用的对象是 `int` 类型。
- `&`：在这里的参数声明中表示引用。
- `x`：函数内部使用的参数名字。

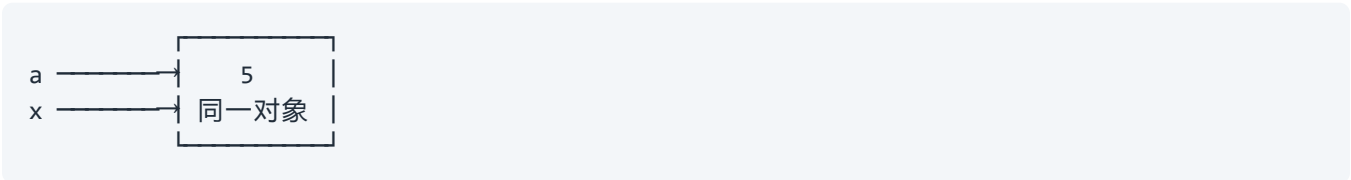
所以 `int& x` 的意思是： **$x$  是一个 `int` 引用参数；调用时，它绑定到所提供的对象。**

2. 对题目逐项作答

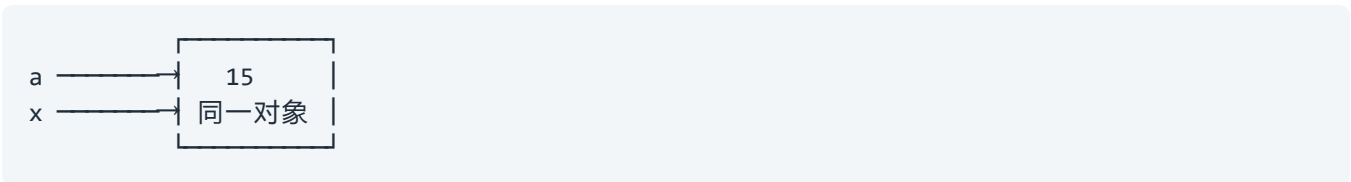
| 问题                             | 答案                                     | 为什么                                   |
|--------------------------------|----------------------------------------|---------------------------------------|
| <code>int&amp; x</code> 是什么意思? | <code>x</code> 是 <code>int</code> 引用参数 | 声明中的 <code>&amp;</code> 表示引用          |
| 调用后 <code>x</code> 引用谁?        | <code>a</code> 所表示的对象                  | 实参为 <code>a</code> , 引用参数绑定到它         |
| 执行赋值后, 谁被修改?                   | <code>a</code> 所表示的那个对象                | 通过引用名字 <code>x</code> 访问的就是同一个对象      |
| <code>a</code> 最后等于?           | 15                                     | 同一个对象从 5 变成 15                        |
| 输出?                            | 15                                     | <code>cout &lt;&lt; a</code> 读取已修改的对象 |

3. 按执行顺序推演

第一步: `int a = 5;` 创建值为 5 的对象。  
第二步: 调用 `change(a)`, 让引用参数 `x` 绑定到这个对象。



第三步: 执行 `x = x + 10;`。右边通过 `x` 读取同一对象的值 5, 算出 15; 左边通过 `x` 把 15 写回同一对象。



第四步: 函数结束。函数内部的引用参数名字不再参与后续执行, 但调用者的对象仍然存在, 值已经是 15。  
第五步: `cout << a;` 输出:

15

这里没有“先改 `x`, 再把结果同步给 `a`”的额外步骤。从绑定完成开始, 通过 `x` 和通过 `a` 访问的就是同一个被引用对象。

4. 两题放在一起, 怎样稳定判断?

| 判断项                         | 题 1: <code>int x</code>          | 题 2: <code>int&amp; x</code> |
|-----------------------------|----------------------------------|------------------------------|
| 参数模型                        | 独立参数对象                           | 绑定到实参对象的引用                   |
| 刚进入函数                       | <code>a = 5, x = 5</code> , 对象不同 | 同一个对象, 值为 5                  |
| <code>x = x + 10</code> 修改谁 | 参数对象 <code>x</code>              | <code>a</code> 所表示的对象        |
| 函数结束后 <code>a</code>        | 5                                | 15                           |
| 最终输出                        | 5                                | 15                           |

## 判断 A：前两个可以构成 overload

前两个声明是：

```
void f(int x);
void f(double x);
```

**可以构成重载。** 它们函数名相同，但参数类型分别是 `int` 和 `double`，当前基础层面关注的签名分别是 `f(int)` 与 `f(double)`。

如果候选只有这两个：

```
f(5);      // int 实参，选择 f(int)
f(5.5);    // double 实参，选择 f(double)
```

判断重载时，比较的是参数类型列表；不是比较参数名字。两个声明都把参数叫 `x`，不会妨碍它们成为不同的重载。

### 原题第三个声明带来的补充

原题还写了：

```
void f(int& x);
```

它和 `void f(int x);` 的参数类型不同，可以作为不同重载同时声明。但是，**重载可以同时声明，不代表每次调用都能选出唯一版本。**

例如，如果原题三个声明都可见：

```
int a = 5;
f(a);
```

`f(int)` 可以接受 `a` 的值，`f(int&)` 也可以绑定到 `a`。在这个调用中，两者没有唯一更优的匹配，所以调用有歧义，不能通过编译。`f(double)` 的存在不会消除这份歧义。

当前先区分两件事即可：**声明能否构成不同重载**，以及**某次调用能否唯一选中一个重载**。本题 A 问的是前一件事，答案是可以。

## 判断 B：不能仅靠返回类型形成 overload

```
int f(int x);
double f(int x);
```

**不能。** 对这里的普通非模板函数而言，函数名相同，参数类型列表也相同，不能仅用 `int` 与 `double` 这两个返回类型把它们区分成合法重载。

调用表达式 `f(5)` 提供的实参信息相同。即使写成下面这样，也不能让原本不合法的两个声明变成合法重载：

```
int a = f(5);
double b = f(5);
```

左边变量想要什么类型，不会使这种“只改返回类型”的普通函数重载成立。不要误判为“把结果接到 `int` 就选 `int` 版，接到 `double` 就选 `double` 版”。

如果想构成重载，可以让参数类型或数量不同，例如：

```
int f(int x);
double f(double x);
```

这里能够区分的原因是参数类型不同。

### 判断 C：不能通过 `const int& x` 给对象赋值

```
void show(const int& x)
```

函数内部不能写：

```
x = 20;
```

这条赋值语句不合法，不能通过编译。

逐层读参数声明：

- `int`：这里访问的是整数。
- `const int`：通过这个受 `const` 限定的访问方式，不能修改该整数。
- `const int&`：通过引用访问对象，并保留上述只读限制。
- `x`：函数内部使用的名字。

函数可以读取 `x`，例如输出它，或者参与计算：

```
cout << x;
int y = x + 1;
```

但是 `x = 20`；要通过这条只读访问路径写入对象，所以不允许。

这不等于原对象永远变成 `const`。例如：

```
int a = 5;
show(a);      // show 的合法函数体只读取 x
a = 20;       // a 本身是非 const 的 int，可以修改
```

### 做这组题时，可以重复使用的解题顺序

1. 找调用表达式，确定实参是什么。
2. 找函数声明或定义，读清参数类型和返回类型。
3. 判断参数是独立对象，还是引用；再判断访问是否受 `const` 限定。
4. 按语句顺序追踪：读取哪个对象、算出什么值、写入哪个对象。
5. 回到调用者，查看哪个对象真正发生了变化，再确定输出。
6. 遇到重载，先检查声明是否合法，再检查当前实参能否唯一匹配；遇到非法代码，说明编译错误，不编造运行输出。

新增解析的语言规则核对依据：C++ 工作草案的 [函数声明与作用域规则](#)、[函数声明规则](#)、[隐式转换序列排序](#)、[const 限定与访问路径](#)。